Codegen—Scaffolding to Get Half the Job Done

The Codegen feature of QCubed creates ORM classes as well as forms for CRUD functionality for all the tables in all the databases included in your project, reducing your database management and querying efforts to half.

The era of static pages is over. No one creates a Web page, but scripts and programs. Almost any site you want to build today will need a database, and for good reasons. Though there are a lot of pre-built packages for various needs like blogging, community forums, e-commerce and so on, every once in a while you will need to do something that cannot be done by one of those packages.

The tediousness lies in working with databases while building something from scratch. Adding more tables and databases to the mix raises the complications and reduces maintainability, exponentially. Listed below are the problems QCubed solves for you through its Codegen faculty. Do note that instead of giving large code samples, I would point you to the examples site (*http://examples.qcu.be/*) that adequately demonstrates the capabilities we are going to discuss. I recommend you do go through the examples to have a better understanding of the QCubed framework.

Note that we have already covered some introduction to Codegen in the previous article. Do refer to it before proceeding.

Writing SQL queries

Let's face the truth: no matter how great you are with SQL, at some point, you will get fed up writing queries, especially the simple ones, which do not test the grey matter, yet consume time. On top of it, adding or modifying columns (renaming, changing data types, etc) would cost you even more time; while confusion often comes along as a free gift.

QCubed creates ORM (Object Relational Model) classes, which reduce your SQL writing task by a (very) large margin. Sure, you can write your own queries, but in most cases, you would not need to. The ORM class created would allow you to load rows, as not arrays, but as *objects* of the ORM class it generates, which are easier in use. Codegen creates the class with Load and *Query* methods for you. The functions that it gifts you are:

- **Load:** Loads one object based on the primary key. The function would accept the number of columns in your primary key. It can accept another parameter, which needs to be a clause object, though in most cases you would not need to use it. This function uses the caching (if you have configured the same in the configuration file) to see if the object was already there in the cache and returns it, otherwise it stores a copy in the cache.
- **LoadAll:** This method will load all the rows of the table as an array of objects (of the table ORM class). Beware, you can easily run out of memory with this one if the table is large!
- **CountAll:** Counts the number of rows in the table.
- **QuerySingle:** This function forms the base for the Load function. The purpose of this function is to run the query on the given conditions and clauses, and then return the first result.
- **QueryArray:** Runs the query on the given conditions and clauses, and returns the result set as an array of objects belonging to the ORM class. I suggest you have a look at a few examples to see how to perform the query—it's a new, interesting and logically easier method.
- **QueryCount:** Returns the number of results that you would get for the query supplied as the argument.